

Artificial Intelligence Concepts and Machine Learning Techniques (AI – 103)

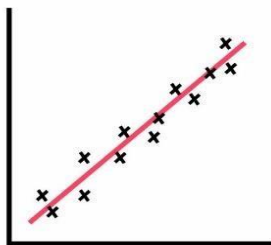
Walaa H. Elashmawi
Professor of AI

 walaa.hassan@miuegypt.edu.eg

Regularization in Regression Models

Correlation

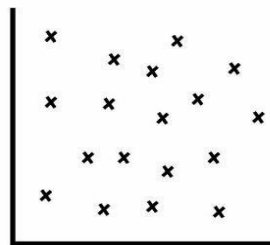
- **Correlation** refers to a statistical measure that quantifies the degree of association or relationship between two or more variables.
- In other words, it helps to determine **whether changes in one variable are related to changes in another variable**.
- Correlation does not imply causation, meaning that just because **two variables are correlated does not necessarily mean that changes in one variable cause changes in the other**.



Positive
Correlation



Negative
Correlation



No
Correlation

A positive correlation is a relationship between two variables that tend to move in the same direction (i.e., when one variable tends to decrease as the other variable decreases, or one variable tends to increase when the other increases).

Correlation Coefficient

- The correlation coefficient is the specific measure that **quantifies the strength of the linear relationship between two variables** in a correlation analysis. It denoted as r
- It ranges from **-1 to +1**. A positive value indicates a positive correlation (as one variable increases, the other tends to increase), a negative value indicates a negative correlation (as one variable increases, the other tends to decrease), and a value close to 0 indicates a weak or no linear correlation.

Sample correlation coefficient represented by
Pearson Correlation Coefficient r :

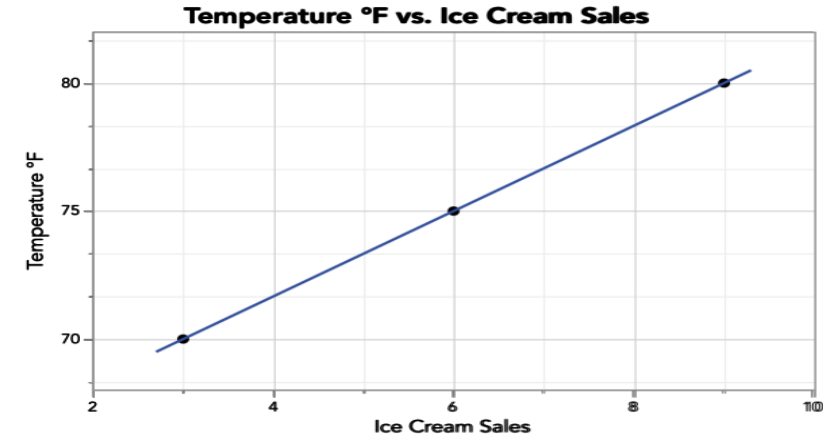
$$r = \frac{\sum [(x_i - \bar{x})(y_i - \bar{y})]}{\sqrt{\sum (x_i - \bar{x})^2 * \sum (y_i - \bar{y})^2}}$$

Diagram illustrating the components of the Pearson Correlation Coefficient formula:

- Value of X** and **Value of Y** are the input variables.
- Summation: "Take The Sum Of"** points to the summation symbol $\sum$.
- Mean of X Variable** points to $\bar{x}$.
- Mean of Y Variable** points to $\bar{y}$.
- Sample Correlation Coefficient** points to r .
- Square Root** points to the square root symbol $\sqrt{\quad}$.
- Sum of the squared deviations for X** points to $\sum (x_i - \bar{x})^2$.
- Sum of the squared deviations for Y** points to $\sum (y_i - \bar{y})^2$.

Example: ice cream sales on hotter days

Ice Cream Sales (X)	Temperature °F (Y)
3	70
6	75
9	80



- Calculate the sample mean

$$\bar{x} = [3 + 6 + 9] \div 3 = 6 \quad \bar{y} = [70 + 75 + 80] \div 3 = 75$$

- Calculate the distance of each datapoint from its mean
- Calculate r

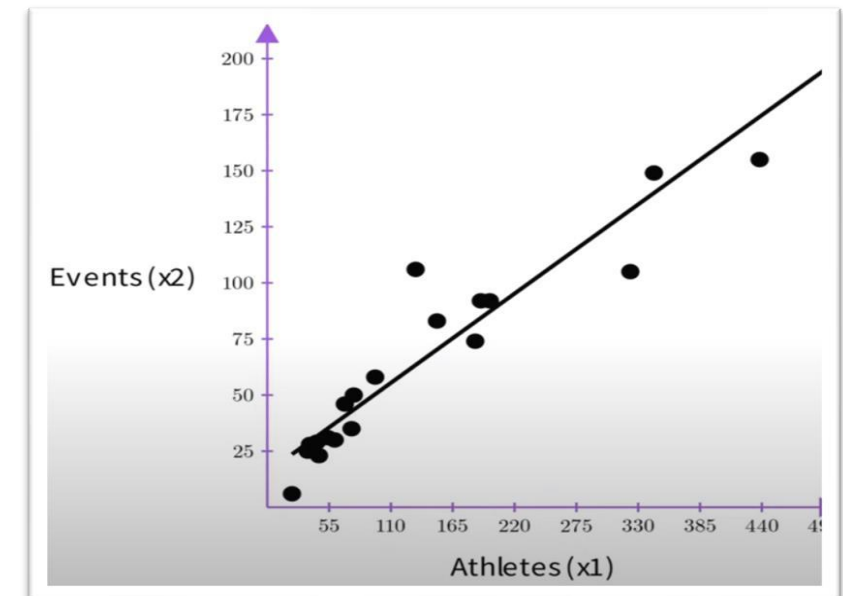
Ice Cream (X)	Temperature °F (Y)	$x_i - \bar{x}$	$y_i - \bar{y}$
3	70	$3 - 6 = -3$	$70 - 75 = -5$
6	75	$6 - 6 = 0$	$75 - 75 = 0$
9	80	$9 - 6 = 3$	$80 - 75 = 5$

$$r = \frac{30}{30} = 1 \quad \text{A perfect correlation between ice cream sales and hot summer days}$$

Multicollinearity

- **Multicollinearity** is a statistical concept that occurs when two or more independent variables (features) in a regression model are **highly correlated with each other**.
- In other words, multicollinearity indicates a strong linear relationship between predictor variables. Two variables are considered perfectly collinear if their correlation coefficient is ± 1.0
- This can pose challenges in regression analysis, particularly in linear regression.

team	athletes	events	medals
SUI	151	83	8
NOR	69	46	17
SWE	198	92	32
JAM	75	35	7



$$medals = f(athletes, events)$$

$$y = f(x_1, x_2)$$

A linear model to predict the electricity bill

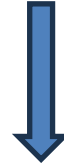
Person	Area (sq. ft.)	Devices	Monthly Bill (\$)
1	1500	8	200
2	1200	5	150
3	1800	10	250
...	...	...	...

Another example

- Suppose you're building a linear regression model to predict a person's monthly electricity bill based on two independent variables:
 1. the square footage of their home (Area)
 2. the number of electronic devices they own (Devices).
- The idea is that both variables could contribute to higher electricity consumption.
- In this scenario, it's possible that larger homes have more electronic devices, leading to higher electricity bills. This correlation between Area and Devices could cause multicollinearity. **When multicollinearity is present, the coefficients of these variables become less stable and harder to interpret** (i.e., The model struggles to decide how much weight to give each correlated variable, which makes interpretation less meaningful).

In other words: →

Multicollinearity causes **unstable, high-variance coefficient estimates** because the model cannot distinguish the individual contribution of highly correlated predictors.



Regularization imposes constraints that improve model robustness and interpretability.

ML Model Error

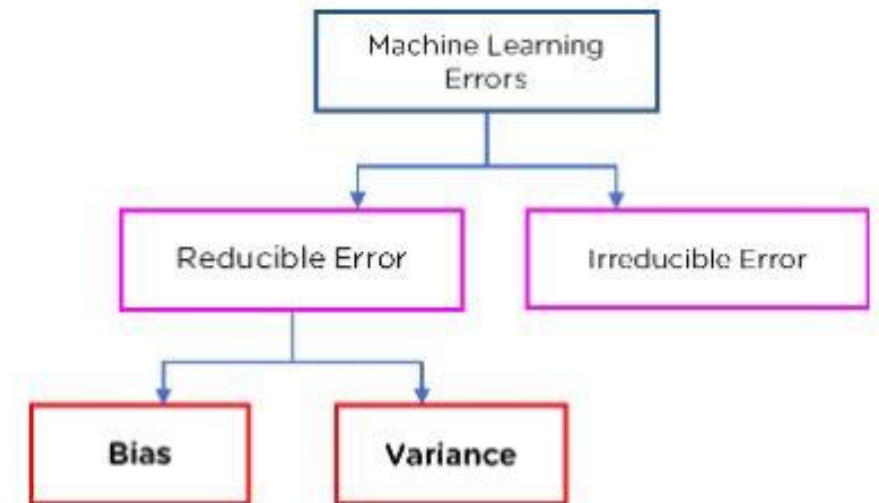
Error: an action which is inaccurate or wrong

- In Machine Learning, error is used to see how accurately our model can predict on data it uses to learn; as well as new, unseen data.
 - ➔ Based on error, we choose the machine learning model which performs best for a particular dataset.

Two main types of errors in ML

✓ Irreducible errors: are errors which will always be present in a machine learning model, because of unknown variables, and whose values cannot be reduced.

✓ Reducible errors are those errors whose values can be further reduced to improve a model. They are caused because our model's output function does not match the desired output function and can be optimized.



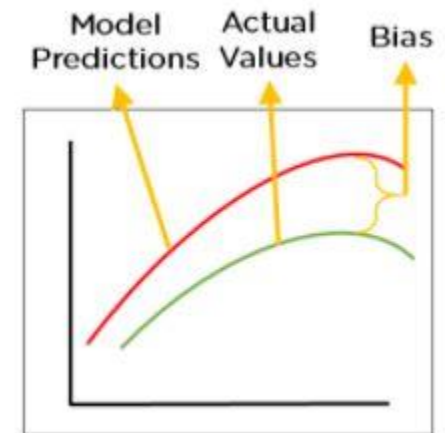
Bias

Basic idea: the model analyzes data to identify patterns, making generalizations about specific instances. After training, it learns these patterns and applies them to the test set for predictions.

- **Bias** is the difference between the average prediction of our model (actual) and the expected value which we are trying to predict (predicted).

Let Y be the true value of a parameter and let $\hat{Y}$ be an estimator of Y based on a sample of data. Then, the bias of the estimator $\hat{Y}$ is given by:

$$Bias^2 = (E[\hat{Y}] - Y)^2$$



Low Bias -> indicates fewer assumptions are made in building the target function, resulting in a model that closely matches the training dataset. ~ it captures the underlying patterns in the data more accurately.



High Bias -> indicates that more assumptions are made in building the target function, resulting in a model that may not closely match the training dataset (underfit). ~ the model is too simple (poor in training and testing).

Ways to reduce Bias:

- **Use a more complex model:** by increasing the number of hidden layers in the case of a deep neural network. Or we can use a more complex model like Polynomial regression for non-linear datasets, CNN for image processing, and RNN for sequence learning.
- **Increase the number of features:** By adding more features to train the dataset will increase the complexity of the model. And improve its ability to capture the underlying patterns in the data.
- **Reduce Regularization of the model:** Regularization techniques such as L1 or L2 regularization can help to prevent overfitting and improve the generalization ability of the model. if the model has a high bias, reducing the strength of regularization or removing it altogether can help to improve its performance.
- **Increase the size of the training data:** Increasing the size of the training data can help to reduce bias by providing the model with more examples to learn from the dataset.

Variance

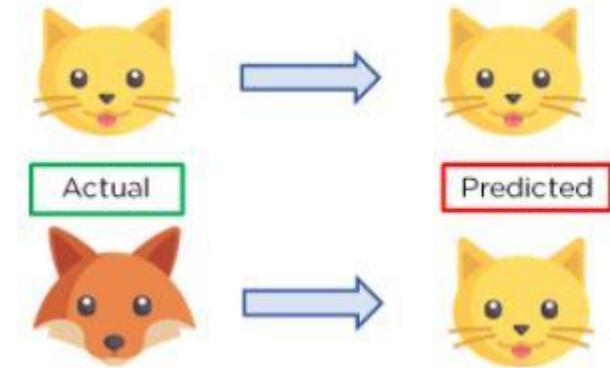
Idea : if our model is allowed to view the data too many times, it will learn very well for only that data. It will capture most patterns in the data, but it will also learn from the unnecessary data present, or from the noise.

Variance is the variability in the model prediction, meaning how much the predictions will change if a different training data set is used.

Let Y be the actual values of the target variable, and $\hat{Y}$ be the predicted values of the target variable. Then the variance of a model can be measured as the expected value of the square of the difference between predicted values and the expected value of the predicted values.

$$Variance = E[(\hat{Y} - E[\hat{Y}])^2]$$

$E[\hat{Y}]$ is the expected value of the predicted values. Here expected value is averaged over all the training data.



Low variance: indicates less sensitive to training data changes and can produce consistent estimates of target function (underfit). ~ generalizes better to unseen data.

High variance: indicates sensitivity to training data changes, leading to overfitting. This occurs when the model performs well on training data but fails on new test data due to overfitting.

Ways to reduce Variance:

- **Cross-validation**: By splitting the data into training and testing sets multiple times, cross-validation can help identify if a model is overfitting or underfitting and can be used to tune hyperparameters to reduce variance.
- **Feature selection**: By choosing the only relevant feature will decrease the model's complexity. and it can reduce the variance error.
- **Regularization**: We can use L1 or L2 regularization to reduce variance in machine learning models.
- **Ensemble methods**: It will combine multiple models to improve generalization performance. Bagging, boosting, and stacking are common ensemble methods that can help reduce variance and improve generalization performance.
- **Early stopping**: Early stopping is a technique used to prevent overfitting by stopping the training of the deep learning model when the performance on the validation set stops improving.

Example

Model 1	Model 2	Model 3	Average Prediction ($E[\hat{y}]$)	Ground Truth (y)
25	29	35	29.67	32
53	52	45	50	47
29	26	24	26.33	26
40	35	37	37.33	36
37	41	33	37	38

Variance Calculation

Variance measures how much the predictions vary around their mean prediction, given by: $\text{Variance} = E[(\hat{y} - E[\hat{y}])^2]$

For the first row: $(25 - 29.67)^2 + (29 - 29.67)^2 + (35 - 29.67)^2$

This calculation is repeated for all rows, summed, and then divided by 15 (the total number of predictions: 5 test points $\times$ 3 models).

Bias Calculation

Bias measures how far the mean prediction is from the true value, given by: $\text{Bias}^2 = E[(E[\hat{y}] - y)^2]$

For the first row: $(29.67 - 32)^2$

This is done for all rows, summed, and divided by 5 (the number of test points).

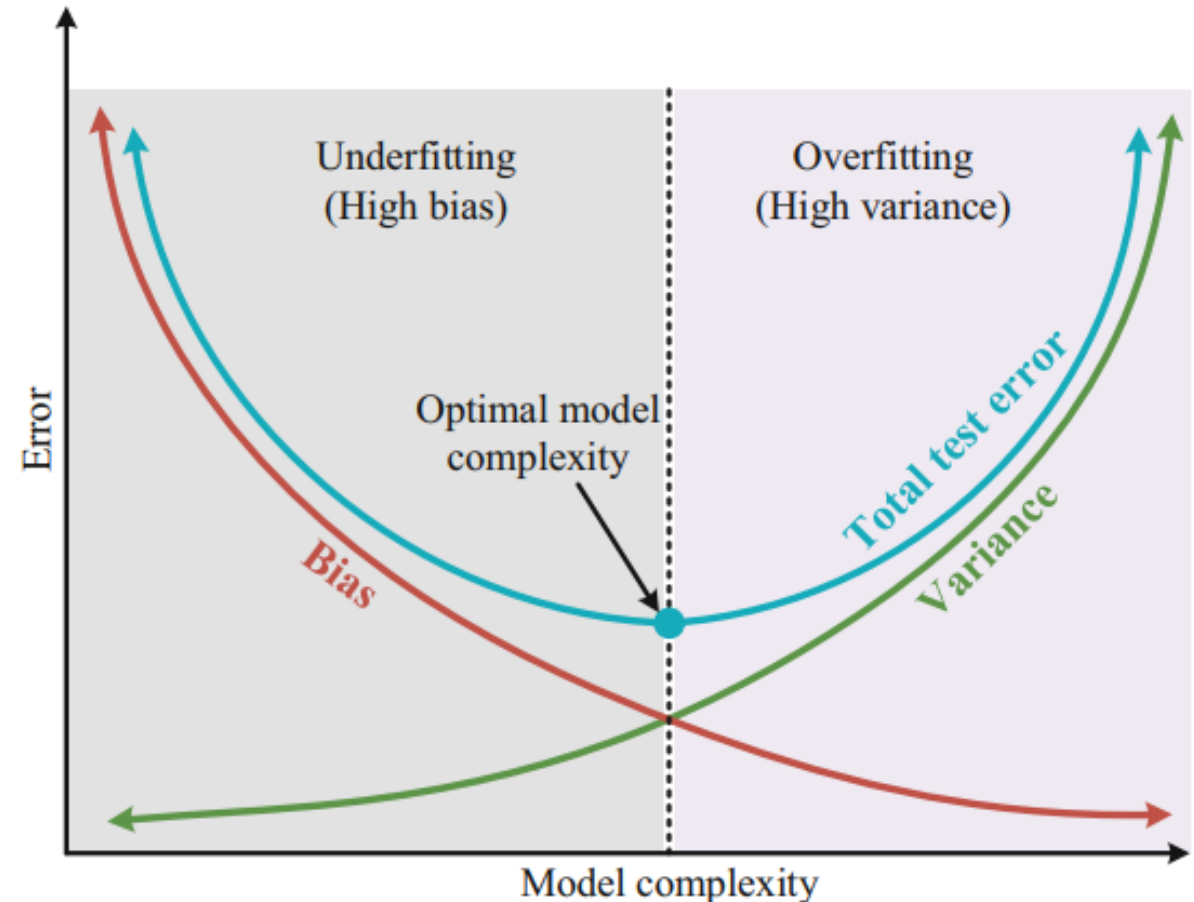
Bias and Variance Tradeoff

If the bias of the model is increased, the variance will decrease. On the contrary, if the model's variance is increased, the bias will decrease.

A low bias and high variance can make a model overfit the training dataset.

A model with high bias and low variance may underfit the dataset.

Therefore, a balance between bias and variance is required to be determined while training the models.

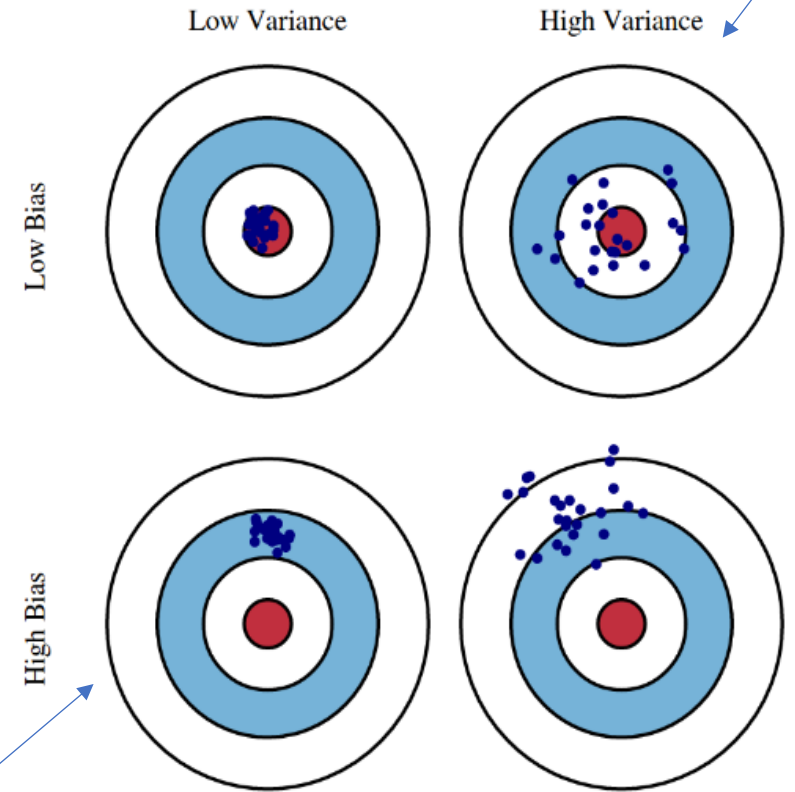


Bias and Variance Tradeoff

- **Low B and Low V:** ideal scenario (it is able to generalize well to new, unseen data and produce consistent and accurate predictions).
- **Low B and High V:** overfitting
- **High B and Low V:** underfitting
- **High B and High V:** The model is not able to capture the underlying patterns in the data (high bias) and is also too sensitive to changes in the training data (high variance). As a result, the model will produce inconsistent and inaccurate predictions on average.

underfitting

overfitting



The center of the target is a model that perfectly predicts the correct values. As we move away from the bulls-eye, our predictions get worse and worse

Ensemble learning is one way to execute this trade off analysis.

ML Model Total Error

Irreducible error (Noise) is unavoidable error caused by elements outside of our control, such as noise from observations.

In general:-

$$Y = f(X) + \epsilon$$

where the error term ϵ is normally distributed with a mean of zero like so $\epsilon \sim N(0, \sigma_{\epsilon}^2)$.

X represents the **input variables** (features, predictors). It can be:

- A single variable (scalar)
- Multiple variables (vector)

$f(X)$ is the **true relationship** between inputs and output.

- Linear: $f(X) = \beta_0 + \beta_1 X$
- Nonlinear: neural networks, trees, kernels

ϵ : **Error / Noise Term** (it captures everything the model does not explain). With assumption, normally distributed

- Noise is **random**
- Noise is **not predictable**
- Noise is **different for every sample**

We never observe $f(X)$ directly — we only observe noisy data.

In the general case, **mean-squared error** can be decomposed into three components: error due to bias, error to variance, and error due to noise.

$$Error = Bias^2 + Variance + Noise$$

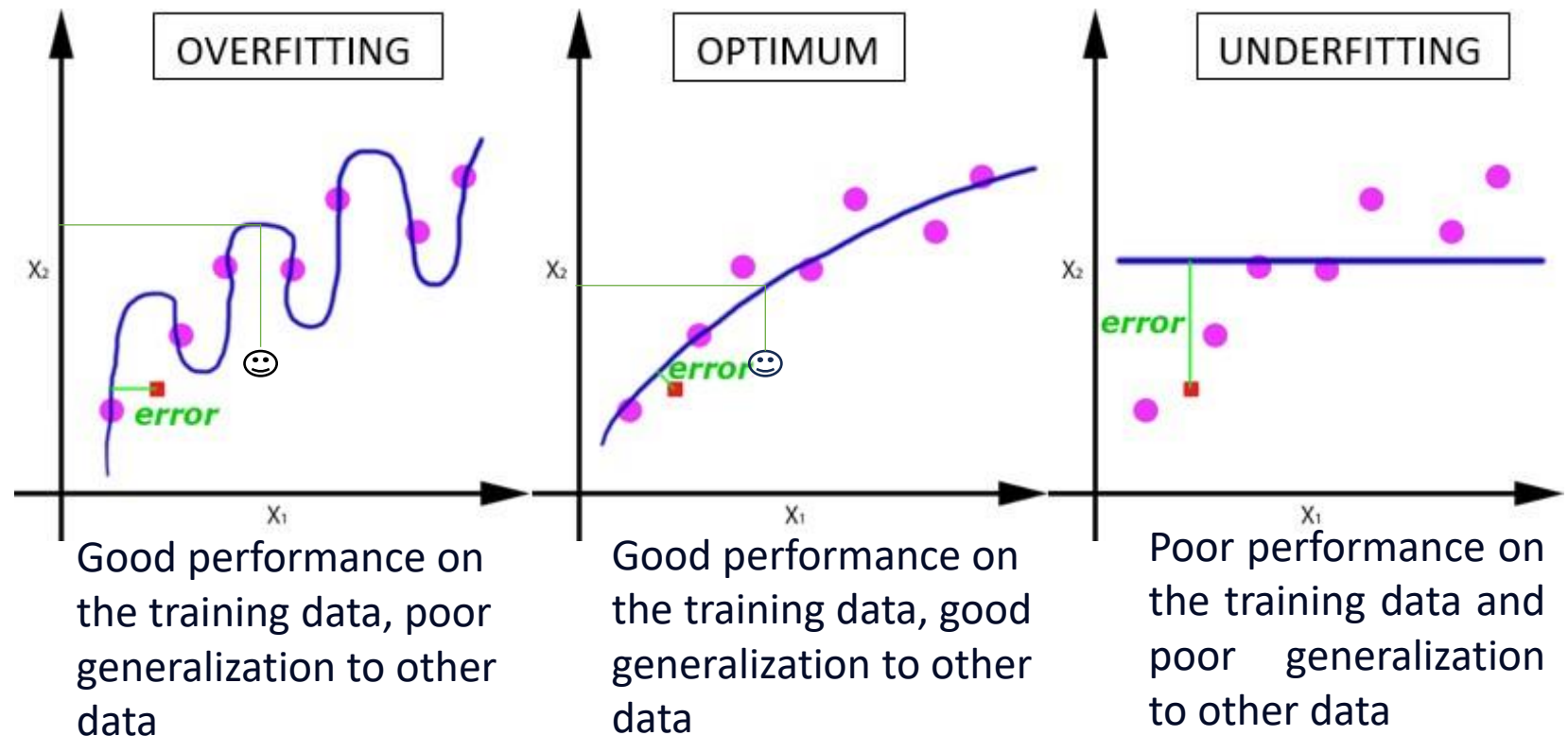
How far is a model from the ground truth ↓	How far is a model from the «average model» ↓	How far is the «average model» from the ground truth ↓	
$E[(\hat{y} - y)^2]$	$= E[(\hat{y} - E[\hat{y}])^2]$	$+ (E[\hat{y}] - y)^2$	$+ \epsilon$
MSE	$= Variance$	$+ Bias^2$	$+ Noise$

Note: Overfitting occurs when a model learns not only the true underlying function $f(X)$ but also, the noise term ϵ , which is random and non-generalizable, leading to low training error but high-test error.

Overfitting

- Overfitting** is a common problem in machine learning and regression models where a model learns the training data too well to the point that it captures the noise and random fluctuations in the data rather than the underlying true relationship between variables.

- In the context of regression models, overfitting occurs when the model becomes overly complex and fits the training data points so closely that it loses its ability to generalize to new, unseen data.



Example: 3 students

Student A attends only a few lectures and barely studies the material.

- He fails to grasp core concepts and rely on minimal knowledge.
- As a result, they perform poorly on both practice tests and the final exam.

The Underfitting Student (Model Too Simple)

Class test ~50%
Final test ~47%

Student B memorizes every question and answer from practice tests instead of understanding the concepts.

- During practice tests, they score perfectly because they have memorized the questions.
- However, in the final exam, when new questions appear, they struggle to apply their knowledge and perform poorly.

The Overfitting Student (Model Too Complex)

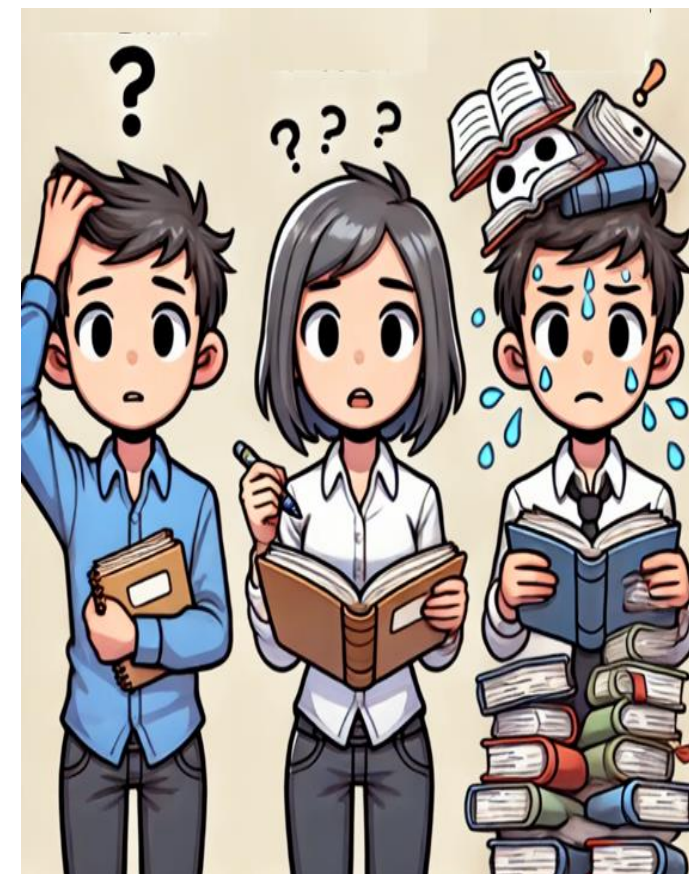
Class test ~98%
Final test ~69%

Student C attends lectures, studies efficiently, and focuses on understanding key concepts.

- They do well on practice tests without memorizing every detail.
- In the final exam, even with new questions, they apply their understanding effectively and perform well.

The Best-Fitting Student (Balanced Model)

Class test ~92%
Final test ~89%



A

C

B



The consequence of overfitting

Ex: If the machine learning model was trained on a data set that contained majority photos showing dogs outside in parks , it may learn to use grass as a feature for classification and may not recognize a dog inside a room.

Poor Generalization

The primary issue with overfitting is that the model's performance on new data, such as a validation set or real-world data, is often significantly worse than on the training data.

Unstable Coefficients

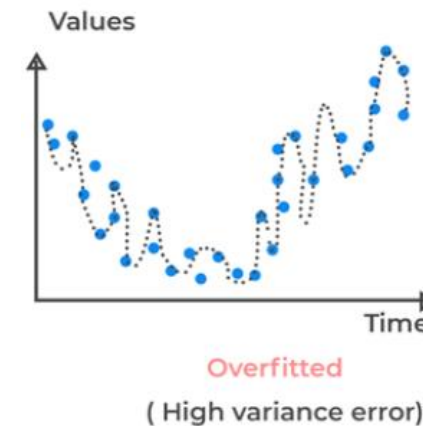
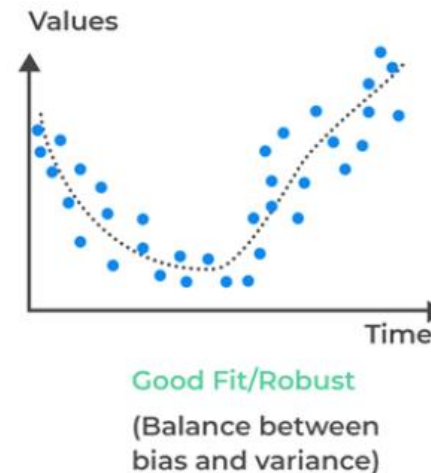
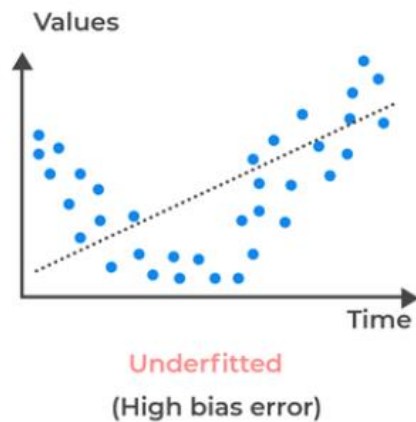
In regression models, overfitting might result in very large or small coefficient values for various features, making it challenging to interpret their impact on the target variable.

Reduced Predictive Power

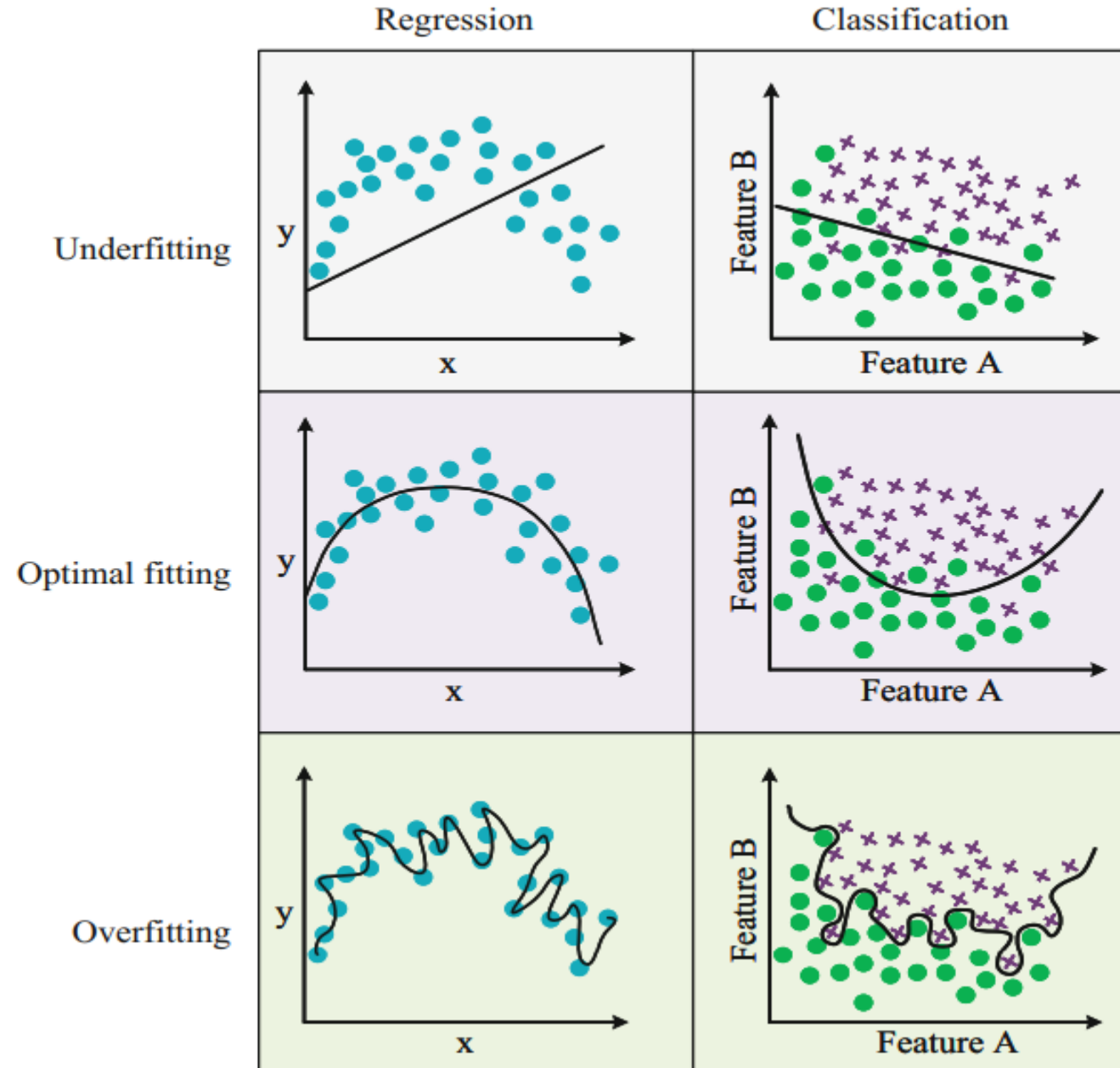
The main goal of a regression model is to make accurate predictions on new data. Overfitting undermines this goal by prioritizing the fit to the training data at the expense of generalization.

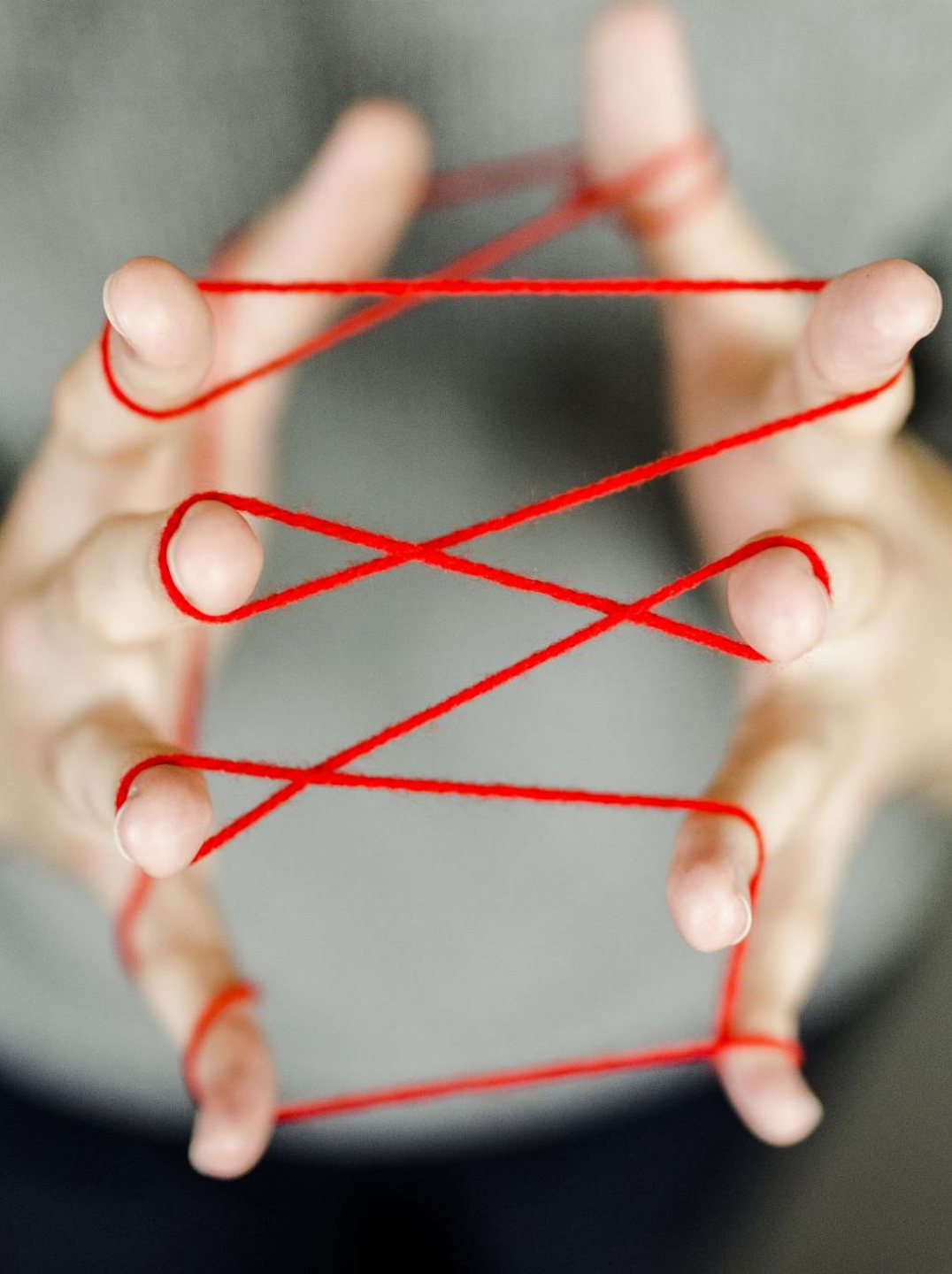
Underfitting Vs. Overfitting

Underfitting	Overfitting
Model is too simple.	Model is too complex.
High bias, low variance.	Low bias, high variance.
Poor performance on both training and test data.	Poor generalization to new data.



The effect of overfitting and underfitting on training data





How to address overfitting?

Regularization

- The cost function is (aka loss function) is a function that calculates the error model generated. *For example*, in linear regression, we usually use Mean Squared Errors (MSE) as a cost function. Regularization tries to decrease or increase the weights in your model to force your model to be simpler.
- Techniques like Ridge Regression and Lasso Regression introduce penalties to the coefficients, discouraging overly complex models.

Regularization

Regularization is a collection of training/optimization techniques that seek to reduce **overfitting** (i.e., The model memorizes every small detail instead of **learning the true pattern**.) and improve the generalization ability of models **by adding constraints or penalties to the model's optimization function**. These methods try to eliminate those factors that do not impact the prediction outcomes by grading features based on importance.



Regularization L1 & L2 **help models to focus on important information** instead of memorizing everything.

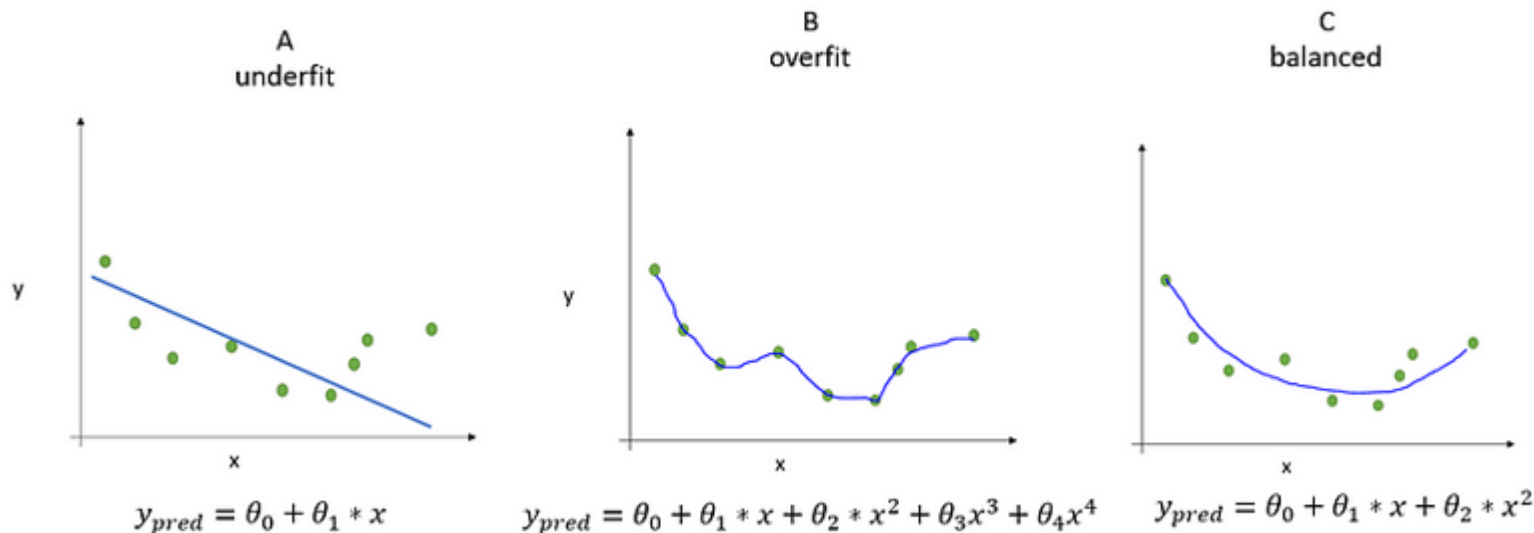
For example, mathematical calculations apply a penalty value to features with minimal impact. Consider a statistical model attempting to predict the housing prices of a city in 20 years. Regularization would *give a lower penalty value to features like population growth and average annual income* but a *higher penalty value to the average annual temperature of the city*.



Regularization

- Regularization technique is based on the fact that if the highest order terms in a polynomial equation have very small coefficients, then the function will approximately behave like a polynomial function with a smaller degree.

- linear regression line fitted in graph A
- higher-order polynomial regression in the B plot (i.e., line is literally a zig-zag that passes through all data points).



C plot: we realize that when we reduce the effect of θ_3 and θ_4 in the equation by making them almost 0. In this way, we get a more balanced line. The idea is here to **shrink** the parameters.

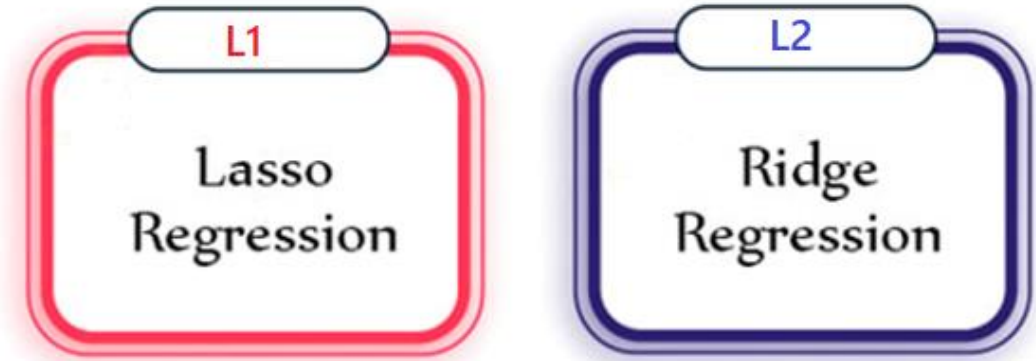
Regularization Techniques in ML

The cost function calculates the error between predictions and actual values

$$\sum_{i=1}^M (y_i - \hat{y}_i)^2 = \sum_{i=1}^M \left(y_i - \sum_{j=0}^p w_j \times x_{ij} \right)^2$$

y_i is the actual value and $\hat{y}_i$ that is the predicted value from our linear equation, where M is the number of rows and P is the number of features.

The most common two techniques



Cost function = Loss + **Regularization term**

The difference between L1 and L2 regularization techniques lies in the nature of this regularization term.

- L1 regularization performs automatic feature selection, identifying the most important features in the dataset.
- In contrast, L2 regularization retains all features in the model and simply reduces their magnitudes.

➤ **L1 Regularization (Lasso):** LASSO stands for Least Absolute Shrinkage and Selection Operator.

- This technique adds a penalty to the **absolute values** of the model's coefficients. This forces certain coefficients to be **equal to zero**. It reduce overfitting and improve prediction accuracy.
- L1 regularization is particularly useful when dealing with high-dimensional datasets where many features may be irrelevant or redundant.
- It is useful for **feature selection**, as it can help to identify which features are most important for the model.

➤ **L2 Regularization (Ridge):**

- L2 regularization adds a penalty to the **squared values** of the model's coefficients. This forces all coefficients to be **close to zero** but does not allow them to be equal to zero.
- It discourages large coefficient values and encourages the model to distribute the importance of features more evenly.
- L2 regularization is effective at reducing the impact of multicollinearity, where features are highly correlated.

Part 1

$$\sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p |\beta_j|$$

Cost function for lasso regression

$$\sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p \beta_j^2$$

Cost function for ridge regression

When train the model, try to minimize cost function (error). Lambda is a penalty parameter which tries to penalize the weights.

Real-Life Examples for L1

Example 1: Hiring for a Job (L1 = strict selection)

- You receive 200 CVs for one position.
- You only care about a **few critical skills** (e.g., Python, ML, experience).
- Everything else (random certificates, irrelevant hobbies)? **Discarded**.
- L1 forces many feature weights to exactly zero.
- Result: only the most important features survive.

Shortlisting Scorecard: Retail

Candidate ID	Education Level	Retail Experience	Customer Service Experience
	(1= min.; 2= desired)	(1= < year; 2= a year; 3= >year)	
Candidate 1			
Candidate 2			
Candidate 3			



Example 2: Cleaning Your Phone Apps

- Your phone has **too many apps** slowing it down.
- You keep only **essential apps** (WhatsApp, Maps, Camera).
- Rarely used apps? **Deleted completely**.
- L1 drives unimportant feature weights to zero.



Real-Life Examples for L2

Example 1: Studying for an Exam (L2 = spread effort)

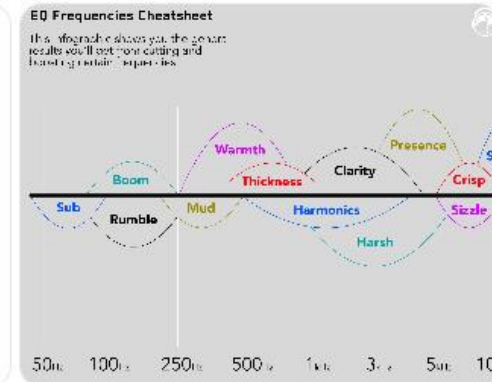
- You have many topics to study.
- You don't focus on just 2 topics and ignore the rest.
- You spread your effort across all important topics, even if some matter more.
- No topic is completely skipped.



STUDY SCHEDULE PLANNER		DATE
		March 12, 2025
		M T W T F S S
TIMETABLE	EXPECTED HOURS	
5 AM	GOAL HOURS	
6 AM	ACTUAL HOURS	
7 AM		
8 AM	SUBJECTS	
9 AM		
10 AM		
11 AM		
12 PM		
1 PM		
2 PM		
3 PM		
4 PM		
5 PM		

Example 2: Adjusting Volume Levels (L2 = smooth balance)

- You're mixing music using a soundboard.
- You don't mute any instrument completely.
- Instead, you turn down loud instruments and slightly boost quiet ones.
- Everything stays, just more balanced.



- ◆ L2 shrinks weights smoothly
- ◆ No feature is removed, just reduced
- ◆ Prevents any single feature from dominating

The effect of λ in Lasso and Advantages of the Penalty term

λ is the **regularization parameter** that controls the strength of the penalty.

- If λ is "small":
Model behaves similarly to ordinary least squares regression (i.e., linear regression).
Coefficients are only slightly penalized.
- If λ is "moderate":
Lasso shrinks some coefficients.
Less important features are eliminated by setting some β_j to exactly zero.
- If λ is "too high":
The penalty term dominates.
All coefficients shrink to zero.
The model loses predictive power and becomes useless.

When λ increases, the bias increases, and the variance decreases and vice versa.

To understand the effect of this additional term "Penalty term," let's assume that the best-fit line is passing through all data points, hence the sum of square error is zero.

The additional term should be minimized to minimize the cost function.

Minimizing the slope means that it makes the line less steep, and this will not make the line pass through all data points, and this will help to prevent over-fitting.

Consider a simple linear regression problem with two features:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2$$

Where:

- y is the target variable.
- x_1 and x_2 are the predictors.

Example data:

x_1	x_2	y
1	2	3
2	3	5
3	5	8

Example

- The objective is to minimize the following least squares loss:

$$\sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_{1i} - \beta_2 x_{2i})^2$$

- This method can lead to overfitting if x_1 and x_2 are highly correlated.

Lasso adds an L_1 penalty to the loss function:

$$\sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_{1i} - \beta_2 x_{2i})^2 + \lambda(|\beta_1| + |\beta_2|)$$

Effect of Lasso:

- λ in Lasso forces some coefficients (β_1 or β_2) to be exactly zero.
- Suppose the model finds: $\beta_1 = 0, \beta_2 = 1.5$
- This means Lasso has removed the influence of x_1 , keeping only x_2 .

Lasso
Regularization
(L1)

Ridge adds an L_2 penalty to the loss function:

$$\sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_{1i} - \beta_2 x_{2i})^2 + \lambda(\beta_1^2 + \beta_2^2)$$

Effect of Ridge:

- Ridge shrinks coefficients but does not make them zero.
- Suppose Ridge finds $\beta_1 = 0.8, \beta_2 = 1.2$
- This means both predictors are considered, but their impact is reduced to prevent overfitting.

Ridge
Regularization
(L2)

Lasso Vs. Ridge

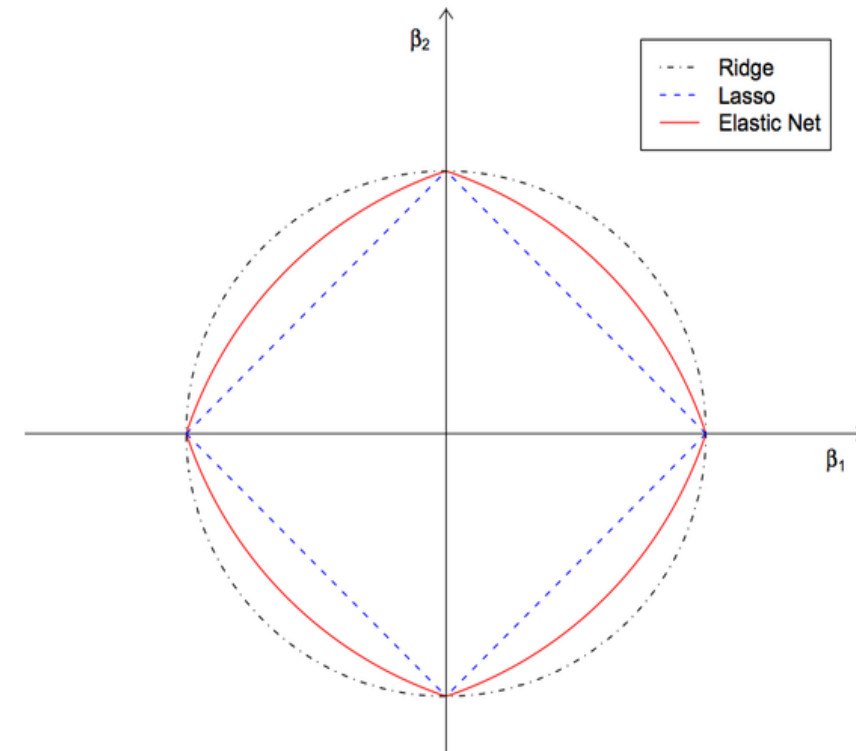


Feature	Ridge	Lasso
Shrinks coefficients	Yes	Yes
Can set coefficients to zero	No	Yes
Suitable for feature selection	No	Yes

- Lasso can lead to sparse models (some coefficients become zero).
- Ridge tends to shrink the coefficients but does not set them to zero.
- Lasso is better for feature selection, while Ridge is better when all features are relevant.

Elastic Net Regularization

- **Elastic Net combines L1 and L2 regularization** to get the benefits of both.
- It applies **two penalties**:
 - **L1 (Lasso)** → performs **feature selection** by driving some weights to zero.
 - **L2 (Ridge)** → **shrinks weights smoothly** to prevent any feature from dominating.
- Elastic Net is especially useful when:
 - Features are **highly correlated**
 - L1 alone is too aggressive (drops useful features)
 - L2 alone keeps too many weak features



Mathematical Form

$$\text{Loss} = \text{MSE} + \lambda_1 \sum |w_i| + \lambda_2 \sum w_i^2$$

- λ_1 :controls sparsity (L1 strength)
- λ_2 :controls smoothness (L2 strength)

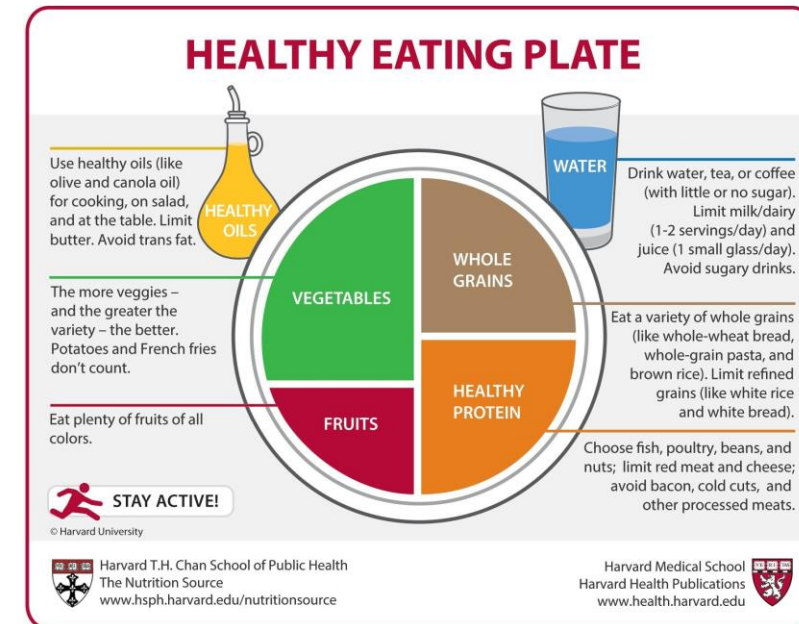
Elastic Net selects important features while keeping correlated features stable.

Real-Life Analogy: Designing a Healthy Diet Plan

- You have **many food items** (features), many of them **nutritionally correlated** (e.g., carbs $\leftrightarrow$ calories, fats $\leftrightarrow$ calories).
- You want a diet that is:
 - **Not overloaded** with unhealthy items
 - **Not too restrictive** to the point of imbalance

How Elastic Net behaves:

- **L1 part:** Completely **removes junk foods** (chips, soda, excess sugar)
- **L2 part:** **Balances portions** of healthy foods (proteins, carbs, fats)
- **Elastic Net:** Drops useless foods & keeps correlated nutrients **in moderation**



Test Your Knowledge

MCQ

Multicollinearity occurs when:

- A) Dependent variables are correlated
- B) **Independent variables are highly correlated**
- C) Target variable is categorical
- D) Noise is zero

Which regularization tends to set some coefficients exactly to zero?

- A) Ridge
- B) **Lasso**
- C) Elastic Net
- D) OLS

Fill in Blanks

- L2 regularization adds a penalty equal to the **square of coefficients.**
- The hyperparameter that controls regularization strength is **lambda (λ)**.
- A model with high bias and low variance is said to be **underfitting**.

THANK
you